

37

PydanticAI

A clean, modern framework for reliable agents

Agents that give structured, predictable output

[AI/ML Engineer Learning Series · Deep Dive Edition](#)

What it is	A Python framework for building AI agents
Best known for	Clean, structured, typed output
Built by	The team behind Pydantic
The win	Reliable, predictable agent results
Feel	Simple, modern, Pythonic

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What is PydanticAI?	An agent framework
2	First, What is Pydantic?	The shape-checking idea
3	The Problem It Solves	Messy LLM output
4	Structured Output	Clean, typed results
5	Your First Agent	A few lines of code
6	Adding Tools	Letting the agent act
7	Why It's Great for Real Apps	Reliability
8	PydanticAI vs CrewAI	Single vs multi-agent
9	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. What is PydanticAI?

PydanticAI is a modern Python framework for building AI agents. After learning what agents are in the last topic, this is one of the clean, practical tools for actually building them. It's made by the team behind Pydantic, a hugely popular Python library, and it brings their signature strength to agents: reliable, structured, predictable output.

Its big selling point is that it makes agents trustworthy for real applications. Instead of an agent returning messy free-form text that's hard to use, PydanticAI makes it return clean, well-shaped data that your code can use directly. For a developer, that reliability is gold.

■ *PydanticAI is a great first agent framework because it's simple and Pythonic. With your MERN and Python background, the clean, typed style will feel natural and professional — a good fit for your portfolio.*

2. First, What is Pydantic?

To understand PydanticAI, know Pydantic first. Pydantic is a Python library that checks data has the right shape. You define what your data should look like — the fields and their types — and Pydantic makes sure any data matches, or raises a clear error.

```
from pydantic import BaseModel

# Define the shape you expect
class Person(BaseModel):
    name: str
    age: int

# Pydantic checks the data fits
p = Person(name='Sam', age=25) # works
p = Person(name='Sam', age='hi') # error! age must be a number
```

This 'define a shape, then guarantee data fits it' idea is what PydanticAI brings to AI agents. You tell it the shape of output you want, and it makes the LLM produce exactly that. No surprises.

3. The Problem It Solves

LLMs return free-form text. That's fine for chatting, but a problem when your code needs to **USE** the output. If you ask an LLM to extract a person's name and age, you might get 'The name is Sam and he's around 25' — nice to read, but a pain for code to pull the actual values from.

PydanticAI gives clean, structured output

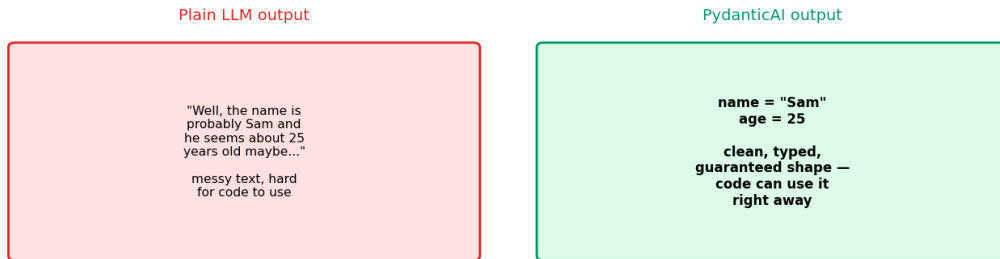


Fig 1 — Plain LLM output is messy text. PydanticAI returns clean, typed data your code can use right away.

Your program wants `name = 'Sam'` and `age = 25` as proper values, not a sentence to parse. PydanticAI solves exactly this: it forces the LLM's output into the clean shape you defined, so it slots straight into your code. No fragile text-parsing, no surprises.

4. Structured Output

This is PydanticAI's headline feature. You define the output shape with Pydantic, and the agent guarantees its answer matches that shape — correct fields, correct types, every time.

PydanticAI: define a shape, get it filled reliably

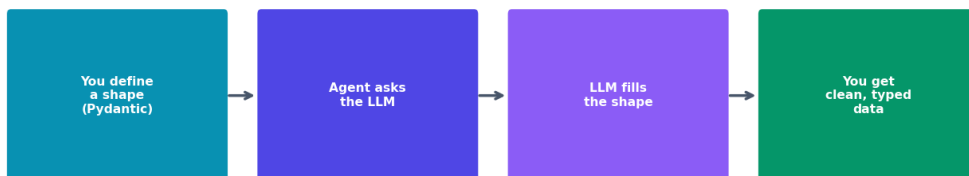


Fig 2 — You define a shape, the agent asks the LLM, and you get back clean, typed data that fits.

Why does this matter so much? Because real apps are pipelines: the agent's output usually feeds into something else — a database, a user interface, or the next step of a workflow. Clean structured output plugs straight in; messy text breaks things.

Clean output plugs straight into the rest of your app



Fig 3 — Structured output flows straight into databases, apps, or the next agent step — no messy parsing.

5. Your First Agent

Here's a complete PydanticAI agent that extracts structured info from text. Notice how the Pydantic shape defines exactly what comes back.

```
# pip install pydantic-ai
from pydantic import BaseModel
from pydantic_ai import Agent

# 1. Define the output shape you want
class PersonInfo(BaseModel):
    name: str
    age: int
    job: str

# 2. Create an agent that returns that shape
agent = Agent(
    'openai:gpt-4o',
    output_type=PersonInfo,
    system_prompt='Extract person info from the text.'
)

# 3. Run it
result = agent.run_sync('Sam is a 25-year-old engineer.')
print(result.output.name)    # 'Sam'
print(result.output.age)     # 25 (a real integer!)
print(result.output.job)     # 'engineer'
```

■ *output_type=PersonInfo is the magic line. It tells the agent: whatever you do, return data in this exact shape. The result is typed and ready to use — result.output.age is a real number, not text.*

6. Adding Tools

Like any agent framework, PydanticAI lets you give your agent tools (Topic 36) — functions it can call to act. You mark a function as a tool, and the agent decides when to use it.

```
from pydantic_ai import Agent

agent = Agent('openai:gpt-4o',
              system_prompt='You help with weather questions.')

# Register a tool with a decorator
@agent.tool_plain
def get_weather(city: str) -> str:
    '''Get the current weather for a city.'''
    return weather_api(city) # your real function

# The agent calls the tool when needed
result = agent.run_sync('What is the weather in Dhaka?')
print(result.output)
```

The agent reads the question, realises it needs weather data, calls your `get_weather` tool, gets the result, and writes the final answer — the Think-Act-Observe loop from the last topic, handled for you. You could register your RAG retriever as a tool the same way.

7. Why It's Great for Real Apps

PydanticAI is built for production, not just demos. Its focus on reliability makes it a strong choice when you're building something real.

Strength	Why it helps
Structured output	Plugs straight into your code
Type safety	Catches errors early, fewer surprises
Works with any LLM	OpenAI, Anthropic, Groq, open models
Clean, simple code	Easy to read, build, and maintain
Tools support	Full agent abilities when needed
Made by Pydantic team	Trusted, well-built foundation

For a developer moving from web work into AI (like you), PydanticAI feels familiar and professional. The structured, typed approach is the same discipline good software engineering already values — now applied to AI agents.

8. PydanticAI vs CrewAI

PydanticAI is great for building one reliable agent. The next topic, CrewAI, focuses on something different: teams of multiple agents working together. Here's the quick contrast.

	PydanticAI	CrewAI
Focus	One reliable agent	Multiple agents as a team
Best at	Structured, predictable output	Agents with different roles
Style	Clean, typed, Pythonic	Role-playing crew of agents
Use when	You need solid single-agent output	You need teamwork between agents

They solve different problems and you can even use them together. Start with PydanticAI to get comfortable building a solid single agent with clean output; reach for CrewAI when your task needs several specialised agents collaborating. Both are modern, friendly frameworks.

9. Common Mistakes

Mistake	Why it's a problem	Fix
Skipping the output shape	Back to messy text	Define an <code>output_type</code> model
Vague field names	LLM fills them wrong	Use clear, descriptive fields
No tool descriptions	Agent misuses tools	Add a docstring to each tool

Over-complex shapes	Hard for the LLM to fill	Keep output models simple
Not handling errors	Agent runs can fail	Wrap runs in error handling
Ignoring the system prompt	Weak guidance	Give a clear system prompt

Quick Reference Cheatsheet

Concept / Task	Meaning or Code
PydanticAI	framework for reliable AI agents
Pydantic	checks data has the right shape
Structured output	clean, typed agent results
Install	<code>pip install pydantic-ai</code>
Output shape	<code>class X(BaseModel): field: type</code>
Create agent	<code>Agent('openai:gpt-4o', output_type=X)</code>
System prompt	<code>system_prompt='...'</code>
Run	<code>agent.run_sync('your input')</code>
Get output	<code>result.output.field</code>
Add a tool	<code>@agent.tool_plain</code> above a function
Tool description	the function's docstring
Best for	single, reliable, structured agents

The one idea to remember: PydanticAI builds reliable agents that return clean, structured, typed output instead of messy text. You define the shape you want, and the agent guarantees its answer fits — perfect for real apps where the output feeds into your code.

Next Topic: CrewAI — a framework for building teams of AI agents that work together, each with its own role, collaborating to complete bigger tasks than one agent could alone.